

DressMe

Phase 4 — Full prototype

AI personal fashion assistant · ESPRIT, Advanced Data Science project (team of 6) · Report generated 2026-10-10

Executive summary

- Phase 4 turns the Phase 3 dataset (*dressme.csv*, 3 public sources, unified schema) into a working app, built as **five sub-projects**: FashionCLIP embeddings, an EfficientNet classifier, a compatibility formula, a FastAPI + MongoDB backend and a React frontend. All five are done, and four layers were built on top of them: listings, AI agents, a local chat model and explainability.
- **FashionCLIP** gives a 512-number vector for 282,135 pictures / crops. A linear probe on these vectors reaches 88.3% category and 79.7% sub_category: the bar the classifier had to beat.
- **EfficientNet-B0** (three heads) beats it on every field: category **95.7%**, sub_category **86.4%**, pattern **86.8%** on the test split. The app uses it for all three.
- The **compatibility formula** (style + colour + pattern + structure, weights set by the team) scores outfits 0-100 with reasons. Swap-test AUC: 66.6% PolyVore, 78.7% Fashionpedia. Style carries almost all the measurable signal.
- **Listings**: clothes for sale now, from two Tunisian shops read politely (Exist, Hamadi Abid), a frozen snapshot of the Inditex shops (which block automated access) and friperie sellers who post in the app after an admin approves them.
- **Five AI agents** answer in the chat, each with its own prompt and tools on the user's real data: Stylist, Shopping advisor, Wardrobe analyst, Seller assistant and Explainer. A router picks one per message.
- **Local chat model**: Qwen3-4B fine-tuned with QLoRA on synthetic chats in French, English and Tunisian Darija runs offline with no daily quota.
- **Explainability (XAI)**: every label, outfit score and buy verdict can say why (heatmaps, confidence and a "not sure, check" flag, points per part, the weakest piece and its best swap). Sub-projects 1-2 of 4 are done, 3 (similarity + chat traces) is in progress.
- **Real-life features**: background removal on every upload, today's weather to choose the season, beach outfits, an Insights page, and a virtual try-on through hosted models with a 2D fallback.
- **Quality**: 293 backend tests with fakes, plus 3 real-model tests; a browser walkthrough of every screen before the jury demo (*DEMO.md*).

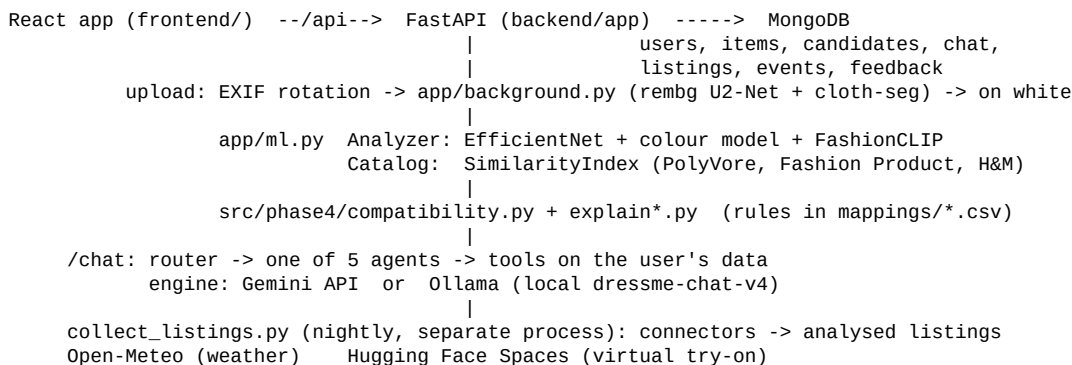
1. Context and goals

Target users are young Tunisians on a limited budget who buy mostly second-hand (friperie): items are often unlabelled and can rarely be returned, so every purchase is a small gamble. The Phase 1 survey (57 responses) set the priorities: 42% want "should I buy this?" advice, 56% shop second-hand, black dominates 84% of wardrobes.

Phase 4 goal: a full prototype that (1) digitises a wardrobe from photos, (2) answers buy / think / skip from one photo in the shop, (3) suggests and scores outfits filtered by modesty, season and occasion, (4) finds look-alikes and pieces for sale, (5) offers a chat assistant that only talks about clothes the user owns, and (6) can explain each of its answers.

Sub-project	Output	Status
4.1 Embeddings	FashionCLIP vectors + similarity index	done
4.2 Classifier	EfficientNet-B0: category, sub_category, pattern	done
4.3 Compatibility	weighted formula, 0-100 score with reasons	done
4.4 Backend	FastAPI + MongoDB, weather, virtual try-on	done
4.5 Frontend	React + Tailwind app and admin dashboard	done
Listings	Tunisian shops, frozen snapshot, friperie sellers	done
AI agents	five chat agents with tools + a router	done
Local chat model	Qwen3-4B + QLoRA, served by Ollama (v4)	done
XAI 1-2	item labels; outfit scores and buy verdicts	done
XAI 3-4	similarity + chat traces; XAI report, Admin > Explainability	done

2. Architecture



Models train on the local GPU (RTX 2050, 4 GB), except the chat model (free Kaggle T4). The backend reuses the `src/` code directly, so the evaluated models are exactly those the app runs. Team-set rules (weights, colour harmony, pattern mixing, outfit structure, weather seasons, XAI cuts, resale prices, router keywords) live in CSV files, never in code.

3. FashionCLIP embeddings

- `src/phase4/embed_fashionclip.py`: one 512-d unit vector (float16) per picture, i.e. per *image_group* for product shots and per item crop (≥ 32 px) for Fashionpedia. 282,135 vectors; resumable, one shard per dataset.
- `src/phase4/similarity.py` (*SimilarityIndex*): nearest-neighbour search, reused by the API for look-alikes, the H&M shop and the listings.
- `src/phase4/evaluate_embeddings.py` → `reports/phase4/embeddings_evaluation.md`.

Results on the 29,633 test pictures

Task	Fashion Product	PolyVore	Fashionpedia	All
Zero-shot category	83.6%	80.8%	62.2%	72.2%
Zero-shot sub_category	70.7%	71.5%	50.4%	60.4%
Linear probe category	96.3%	94.8%	80.8%	88.3%
Linear probe sub_category	88.1%	93.6%	69.1%	79.7%
Linear probe pattern	72.4%	—	74.4%	74.2%

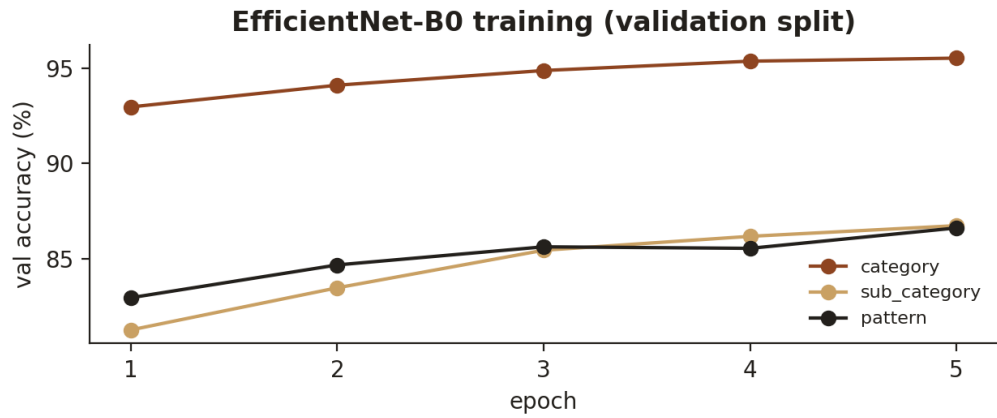
- **Colour:** on hand-labelled items the vectors do worse than the Phase 3 gradient-boosting model (PolyVore 63.1% vs 76.7%, Fashionpedia 42.4% vs 65.9% at the 0.7 cut), so the gradient-boosting colour model stays.
- **Street → shop retrieval:** for 300 Fashionpedia crops, 65% of the 5 nearest product shots share the street item's category.
- The probes are trained on the GPU (*Probe* class): sklearn's logistic regression was far too slow on this machine.



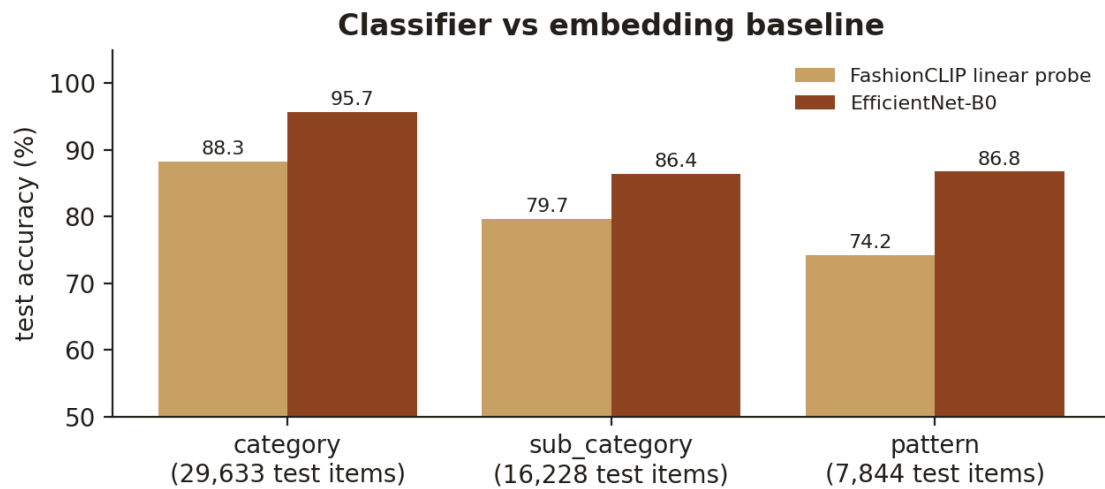
Street photo crops (left) and their nearest product shots.

4. EfficientNet classifier

- EfficientNet-B0 with ImageNet weights and **three heads** (category, sub_category, pattern). Each picture only trains the heads it has a label for; classes are weighted $1/\sqrt{\text{freq}}$.
- `src/phase4/build_image_cache.py` packs every picture, letterboxed to a 224 px white square, into one file: opening 282k small files on Windows ran at ~300 img/s.
- `gpu_batch` decodes JPEGs and augments on the GPU (no hue change, so colours stay true). Training: 5 epochs, ~20 min each (~102 min total) at ~200 img/s, resumable.
- `predict` keeps the sub_category inside the predicted category, and also returns the top 3 answers per field for the explanations (section 10).



Validation accuracy per epoch (reports/phase4/classifier_training_log.csv); the best epoch is the last one.

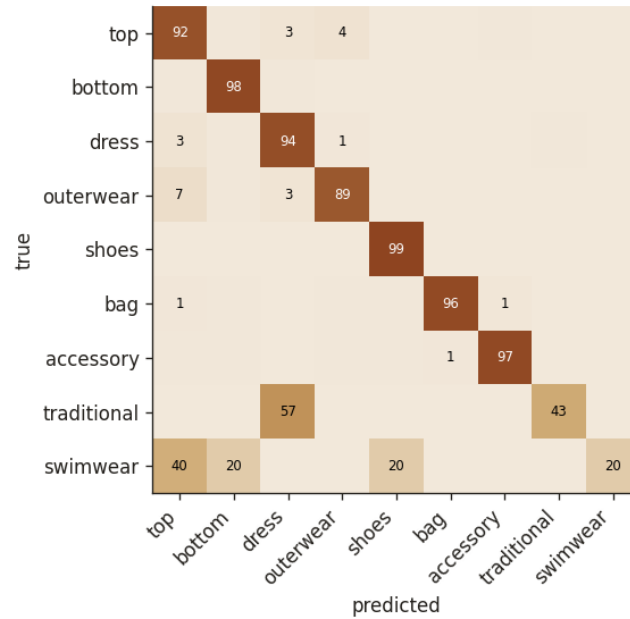


Same 29,633 test pictures for both models. Team rule: the app uses whichever is better per field.

Field	Fashion Product	PolyVore	Fashionpedia	All	Balanced
category	98.6%	97.7%	93.2%	95.7%	80.9%
sub_category	89.2%	94.3%	81.2%	86.4%	79.8%
pattern	82.9%	—	87.1%	86.8%	77.0%

The biggest gain is on Fashionpedia street photos (category 93.2% vs 80.8%, sub_category 81.2% vs 69.1%): fine-tuning learns to read crops of worn clothes. **Traditional** (3/7 right, mostly read as dress) and **swimwear** (1/5) are too small to learn, which explains the lower balanced accuracy.

EfficientNet: category confusion on test (% of each true class)



Category confusion matrix on the test split.

5. Compatibility formula

src/phase4/compatibility.py: score = weighted sum of four parts from 0 to 1, shown as 0-100 with reasons. A part that cannot be computed (e.g. no colour known) is left out and the other weights are rescaled.

Part	Weight	How it is computed
style	0.35	mean FashionCLIP similarity between items, rescaled 0.20 → 0, 0.80 → 1
colour	0.30	colour-group pair table (colour_harmony.csv) + penalty above 3 bold colours
pattern	0.15	worst clothing pattern pair (pattern_mixing.csv); predictions under 0.6 confidence ignored
structure	0.20	top + bottom or a full piece, plus shoes, per-category limits (outfit_structure.csv) and the worst sub_category pair, e.g. blazer + track pants (sub_category_pairing.csv)

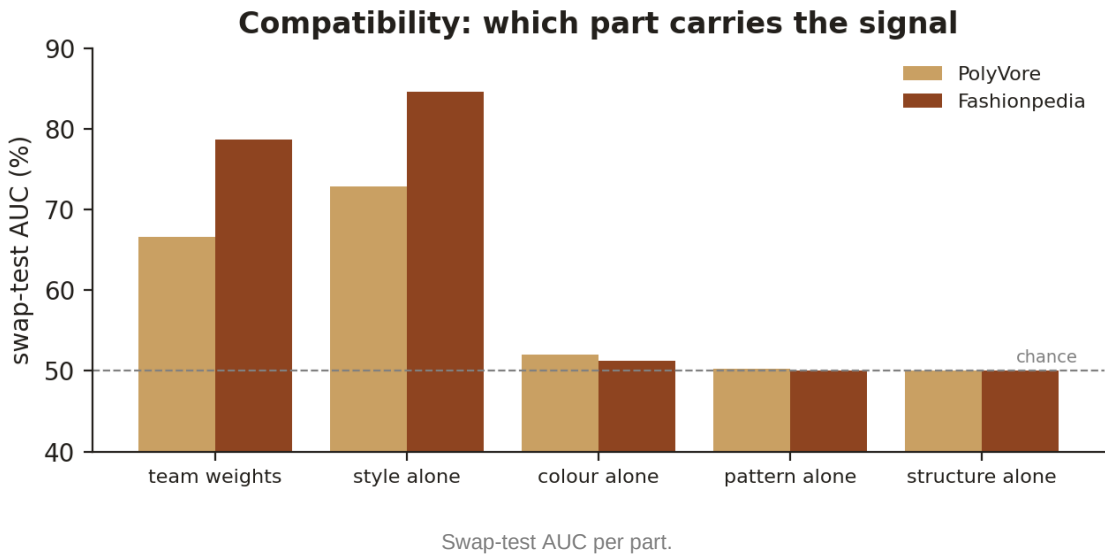
- **Functions:** *filter_items* (profile: min_coverage, season, occasion; unknown fields pass), *suggest_outfits*, *complete_outfit*, *buy_advice* and *wardrobe_insights* (good outfits, most versatile and unmatched pieces, what is missing, near-twins).
- **Seasons:** an item without its own season gets a default one (*item_seasons.csv*: shorts in summer, coats not in summer), applied softly to tops, bottoms and shoes so an outfit is still found. **Beach:** outfits built around a swimsuit, without shoes, always as summer; swimwear is left out of everyday outfits.
- **Hard rule (clashes):** an outfit never holds the same sub_category twice (two pairs of jeans) or more items of a category than *outfit_structure.csv* allows. *complete_outfit* never proposes a clash, and scoring one answers 422.
- **Should I buy this?** Counts the good outfits (score ≥ 60) where the new item beats every owned item of its category: ≥ 5 → buy, ≥ 1 → think, else skip. It also flags near-twins already in the wardrobe (similarity ≥ 0.9).
- **Weights and rules are set by the team;** the data only measures them. They are still marked REVIEW until the team tunes them, and admins can edit them live from the dashboard.

Evaluation

AUC = how often a real outfit beats the same outfit with one item swapped for a random item of the same category (50% = chance). FITB = finding the hidden item among 4 of the same category (25% = chance).

Weights	PolyVore AUC	PolyVore FITB	Fashionpedia AUC	Fashionpedia FITB
team weights	66.6%	42.6%	78.7%	59.5%
style alone	72.9%	49.7%	84.6%	70.0%
colour alone	52.0%	28.5%	51.2%	27.2%
pattern alone	50.3%	25.5%	50.0%	25.1%
structure alone	50.0%	25.0%	50.0%	25.0%
learned reference	72.7%	49.2%	84.6%	70.1%

904 clean PolyVore test outfits and 2,710 Fashionpedia test photos. A model fitted on the data puts ~0.95 on style and ~0.05 on colour. Colour, pattern and structure add no signal in the swap test: structure cannot by design (the swap keeps the category), and real outfits are not chosen for colour rules alone. They are kept because they give the user readable reasons and enforce wearable outfits; the trade-off is a team decision.



Outfits suggested from a mock wardrobe of 40 PolyVore test items.

6. Backend: FastAPI + MongoDB

`backend/`, run with `uvicorn app.main:create_app --factory`; OpenAPI docs at `/docs`. Local MongoDB 4.0 (pymongo pinned below 4.14).

Endpoints	Purpose
<code>/auth/register</code> , <code>/auth/login</code> , <code>/me</code>	email + password (bcrypt), JWT; profile: name, min_coverage, language
<code>/items</code> (POST, GET, PATCH, DELETE), <code>/items/{id}/image</code>	upload → analysed (category, sub_category, pattern, colour, vector, top-3 answers). PATCH stores user corrections in <i>corrected</i> ; the model's guesses stay in <i>predicted</i>
<code>/analyze</code> , <code>/buy-advice</code>	shop candidate (not saved, deleted after 24 h) and its buy / think / skip verdict
<code>/outfits/score</code> , suggest, complete, explain, limits	the compatibility formula on the user's wardrobe, with points and explanations; 422 on a clash; beach outfits
<code>/similar</code> , <code>/catalog/{id}/image</code>	look-alikes in the wardrobe, the public catalogue, the H&M shop and the listings in stock
<code>/insights</code>	what the wardrobe makes, lacks and holds twice
<code>/listings</code> , <code>/listings/sell</code> , <code>/listings/mine</code>	browse listings, sell a piece (admin approval first), manage one's own listings
<code>/chat</code> , <code>/chat/history</code>	five agents with tools (section 8); Gemini or the local model (<i>CHAT_ENGINE</i>)
<code>/items/{id}/explain</code> , <code>/candidates/{id}/explain</code>	why these labels (heatmaps, computed on demand, never stored)
<code>/weather</code> , <code>/tryon</code>	today's weather and season; a picture of the user wearing the chosen pieces
<code>/admin/*</code>	usage stats, model quality (how often users correct each field), users, live editing of the formula, listing sources, moderation, collector runs

- **Clean photos before analysis** (`app/background.py`): every upload is turned upright with its EXIF rotation tag, shrunk to 1024 px, its background removed with rembg (U2-Net) and pasted on white, cropped around the item. When the item is worn, U2-Net cloth-seg cuts it down to the biggest garment. If the mask finds under 3% of the photo, the original is kept. ~3-5 s per photo on the CPU.
- **Weather** (`app/weather.py`): Open-Meteo (free, no key), cached 30 min per place. The season to dress for comes from the day's feels-like temperatures with the team's thresholds (`weather_seasons.csv`, REVIEW). Positions are rounded to ~10 km and never stored.
- **Virtual try-on** (`app/tryon.py`): the try-on models (CatVTON, Kolors, IDM-VTON, OOTDiffusion) need 8-16 GB of GPU, so the picture comes from Hugging Face Spaces, tried in order; one garment per call, an outfit is chained (at most 3). The person photo goes to the Space (the app says so) and nothing is stored. If no Space can dress the garments, the app falls back to a 2D overlay.
- **Privacy**: every query is filtered by `user_id`; another user's item answers 404. Agent tools are bound to the user, and a seller's contact never reaches the language model.
- **Honest colour**: a colour below 0.7 confidence is left empty (the guess is kept in *predicted*) and the UI asks the user to confirm it.
- **Tests**: `python -m pytest in backend/` — 293 fast tests with fake models, a fake background remover, fake weather and recorded shop answers on a `dressme_test` database, plus 3 real-model tests (`DRESSME_SLOW=1`).

7. Listings: shops and friperie sellers

"Should I buy this?" is more useful when the app also knows what is for sale. `mappings/listing_sources.csv` (team-owned) lists the sources; `src/phase4/collect_listings.py` runs nightly as a separate process: each enabled and approved source's connector reads the products, and every new picture is analysed like an upload. Products missing from a complete run become *gone*. Details in *LISTINGS.md*.

Source	How	Status
Exist (exist.com.tn)	sitemap → product pages' schema.org data, through the polite client (robots.txt, delay, stop on 403 / 429)	running
Hamadi Abid (ha.com.tn)	its sitemap, then the JSON call its own pages make (adult sections only)	running

Source	How	Status
Zara, Bershka, Pull&Bear	Access Denied on the first page (bot protection), again on a polite re-check on 2026-10-07: shown only as a frozen, dated snapshot	off
H&M France	every product page answers 403 (bot protection)	off
Zen	robots.txt forbids its product pages	off
Friperie sellers	photo + price + size + city + contact in the app; pending until an admin approves it	running

- **We never work around a block:** a refusal stops the source, saves nothing and shows on Admin > Sources. Before a source is enabled, a read-only check (*check_shop_source.py*, or the Check button) detects its kind and a team member reads its terms.
- Products are saved one at a time as they are read, so a stopped or blocked run keeps what it saved; only a complete good run marks products gone.
- **In the app:** Shops (browse, then "Should I buy this?" on any listing), Sell (post and manage one's listings), an "In shops now" row in Similar, and Admin > Listings (counts, charts, collector runs started, followed live and stopped).
- Labels on listings are model predictions only; prices in EUR are converted with the team's rates (*currency_rates.csv*, REVIEW).

8. AI agents in the chat

The chat is five agents. Each one is a system prompt (the shared rules: answer in the user's language, never invent clothes, respect the modesty level, be short) plus its own tools: Python functions bound to the current user, which the model calls and the backend runs. Every agent also has *list_wardrobe*. Details in *AGENTS.md*.

Agent	Helps with	Its own tools
Stylist	what to wear today or for an occasion	suggest_outfits, score_outfit, complete_outfit, get_weather
Shopping advisor	should I buy this, where to find a piece	buy_advice_last_scan, search_listings, find_similar
Wardrobe analyst	what is missing, unused pieces, duplicates	wardrobe_insights, wardrobe_stats, find_near_twins
Seller assistant	what to sell and at what price	pieces_to_sell, price_hint, my_listings, prepare_sell
Explainer	why a score, a label or a verdict	explain_outfit, what_if, explain_labels, explain_verdict, how_scoring_works

- **Router** (*agents/router.py*): one short model call picks the agent; if it fails, the team's keyword table decides (*agent_keywords.csv*, REVIEW, Darija rows need the native review); else the Stylist. *ROUTER=keywords* skips the call to save Gemini quota.
- **Advise, never act:** the Seller assistant prepares a filled Sell form as a button; the user adds their contact and sends it, and an admin approves it. Prices come from look-alike listings (shop prices × the team's resale factor, REVIEW).
- Every answer carries its agent (a badge in the app) and the pictures of the pieces its tools returned; the Explainer can open the labels panel of a piece.
- **Engine** (*app/chat_engine.py*): Gemini (free key: 20 requests a day, one answer costs 2-7) or a local model with Ollama. The backend runs the tool calls (at most 6 rounds); a model that repeats the same call is asked to answer without tools, and an empty answer becomes a short message in the user's language.

9. Local chat model

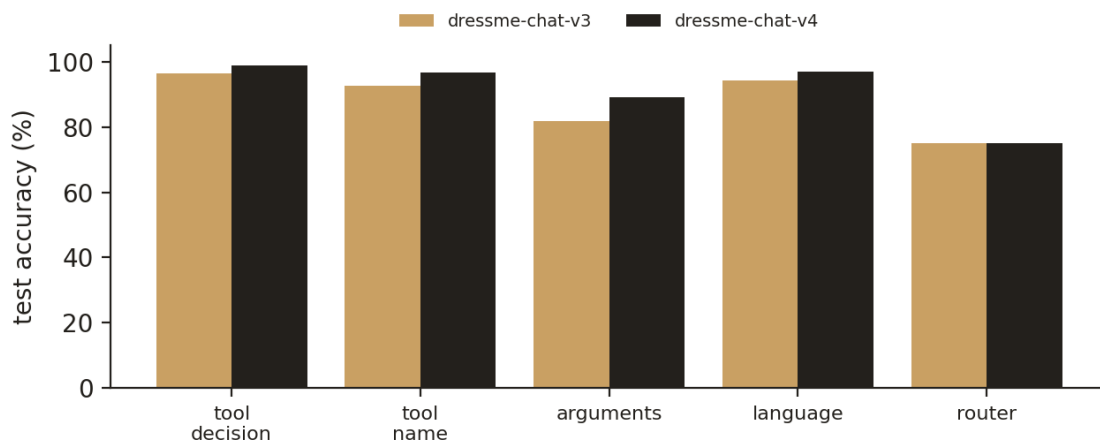
With `CHAT_ENGINE=ollama` the agents run on our own machine: no key, no daily quota, and the demo works offline. The model is Qwen3-4B-Instruct, fine-tuned with QLoRA (Unsloth, rank 16, only the assistant's turns learned) on a free Kaggle T4, exported to GGUF and served by Ollama. Details in *LLM.md*.

Step	What
Dataset	<i>build_chat_dataset.py</i> : 4,500 / 250 / 400 synthetic chats (French 45%, Darija 35% incl. Arabizi, English 20%), each with one agent's real prompt and tools, plus ~5% router rows. The backend's real tools run on random wardrobes and listings in a scratch database, so every score, price and explanation in the answers is real.
What it teaches	list the wardrobe before any tool that takes ids, check the weather before "what do I wear today?", no tool for greetings or off-topic, send a question for another agent to it in one sentence, and say so when the user names a piece they do not own. 40% of the chats start with earlier turns as plain text, as the app sends them: seen by the model but not learned
Training	v4: 220 min on the T4, 1 epoch
Darija	<i>reports/phase4/darija_review.md</i> : every Darija word and sentence for a native check; the agents' new phrases are not reviewed yet

Evaluation on 150 test chats (343 decisions per model: each assistant turn is one decision, the model sees the real conversation up to that point; the test wardrobes and a third of the phrasings were never seen in training):

Model	Tool decision	Tool name	Arguments	Language	Router	s / decision
dressme-chat-v3	96.5%	92.6%	81.8%	94.3%	75.0%	5.8
dressme-chat-v4	98.8%	96.6%	89.2%	96.9%	75.0%	5.8

Chat models on the same test conversations



Same test decisions for both models (v4 test set). The router column rests on only 4 router decisions.

This evaluation is lenient: the model sees the **correct** conversation up to each decision, so once the right wardrobe list is there the next tool is easy. *evaluate_chat_e2e.py* runs **whole chats** through the app's engine and real tools with no help (*reports/phase4/chat_e2e_evaluation.md*):

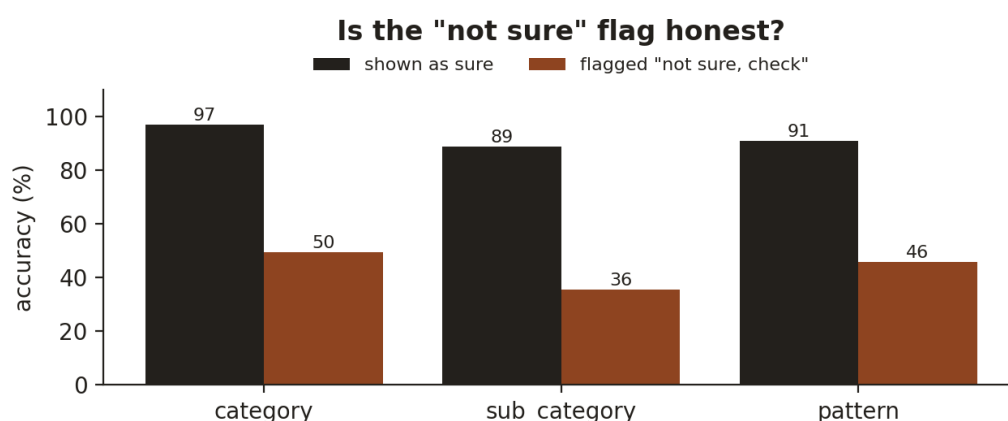
Whole chats, no help	dressme-chat-v3	dressme-chat-v4
chats with earlier turns (history)	33 / 47	39 / 47
chats without history	41 / 51	41 / 51
sell another piece (after a sale)	5 / 8	8 / 8
hand-offs to another agent	3 / 8	5 / 8
scoring a piece the user does not own	4 / 6	1 / 6
all	74 / 98 (75.5%)	80 / 98 (81.6%)

- **An audit found why v1 failed:** it was trained on the original single assistant (4 tools) while the app now sends five agents and 20 different tools, and the chat template shipped with the base model wrote an empty `<think></think>` before every last answer, which the app's template never does. v1 learned it, began its answers with stray tags and called tools for "thank you". Both are fixed since v2.
- **v3** added the Explainer's look-alike tool (v2 made up a reason in a live chat), the app's own words in search (v2 wrote *slacks*, *violet*) and what to do with a piece the user does not own. **v4** (the app's model, team rule: the better one) fixes a failure found in a real chat: the app sends earlier turns as plain text, without the tool calls behind them, and v2 / v3, trained only on chats whose earlier turns kept their tool calls, then answered from that text ("I can't find a blue jacket" the wardrobe holds). v4's data has such history, shown to the model but hidden from the loss (*finetune_chat.mask_history*).
- **Still weak in v4:** scoring a piece the user does not own (1 / 6: it scores another piece instead, a regression from v3's 4 / 6), and hand-offs (5 / 8). The test chats come from the same generator as the training data: they measure tool use and language, not how natural the answers sound with real users.
- **Speed** on the RTX 2050 with the backend's models loaded: ~21 tokens/s, 11-18 s per answer that uses tools (Ollama puts 58% of the model on the GPU), plus ~3 s for the router's call.
- When a tool or an agent's job changes, the dataset is rebuilt and the model retrained: it only knows the tools as they were when it was trained.

10. Explainability (XAI)

Users act on the app's answers with their own money, so every answer can say why. The explanations only show how an answer was made; they never change it. Built as four sub-projects (specs and plans in *docs/superpowers/*); the methods, the faithfulness tests, screenshots, limitations and open team decisions are in the separate **XAI report** (*reports/phase4/xai_report.pdf*).

Sub-project	What the user sees	Status
1. Item labels	"Why these labels?": where the model looked (Grad-CAM), the colour's pixels, the top 3 answers and a "not sure, check" badge	done
2. Outfits and verdicts	points per part that add up to the score, what works and what does not, the weakest piece and its best swap; the path to a buy / think / skip verdict	done
Explainer agent	the same explanations in the chat, plus "what if I wear X instead?" and "why are these two alike?"	done
3. Similarity + chat	why two pieces look alike (shared labels, concepts); "How I answered" under each chat answer	done
4. XAI report + admin	the jury report and Admin > Explainability (evaluations, real-use flag, chat tool calls, XAI settings)	done



Test split, 29,633 pictures. Answers flagged "not sure" are right about half the time or less, so the flag points the user at the labels worth checking. More results in the XAI report.

11. Frontend: React + Tailwind

frontend/: React 19, Vite, TypeScript, Tailwind 4, talking to the API through the */api* dev proxy.

Screen	What the user does
Today	outfit of the day for today's weather (or a chosen season and occasion, incl. Beach), with "Why this score?"
Scan	photo in the friperie → analysis + buy / think / skip, "Why this verdict?", try it on
Wardrobe, Insights	add items by photo, confirm or correct the labels ("Why these labels?"); what the wardrobe makes and lacks
Build	pick pieces, get a 0-100 score with its points, apply the best swap, let the app complete the outfit, try it on
Similar	"you already own something like this", H&M and listings look-alikes
Shops, Sell	listings in stock with their verdict; post and manage one's own friperie listings
Chat	the five agents, with a badge, item pictures and action buttons
Profile	name, modesty level (<i>min_coverage</i>), language
Admin	stats, model quality, users, formula weights, listing sources, moderation, collector runs

- Three languages: English, French and Arabic (right-to-left), with plural rules; the explanation lines are built from codes and translated in the app.
- Design rules in *DESIGN.md*, product brief in *PRODUCT.md*.

12. Testing and demo readiness

Before the jury demo the whole app was run as a user would meet it: the real API, MongoDB and the React app in Chromium, at phone and desktop widths, with the test fakes standing in for the models and Gemini. The walkthrough, the chat audit and the work that followed found and fixed these bugs:

Found	Fix
Build page showed the score panel twice on phones	the desktop side panel is hidden below the desktop width
Phone portrait photos were analysed lying on their side (EXIF rotation ignored)	uploads apply the rotation tag; a test fails without the fix
<i>merge_and_split.py</i> crashed on pandas 3 (allowed by <i>requirements.txt</i>)	one column type fixed; output byte-identical to pandas 2
The local chat engine stopped a normal plan (list tops, list bottoms, score) before scoring, and replaced an empty answer with a made-up English sentence	only a repeated identical call stops it; an empty answer becomes a short message in the user's language
The chat model v1 began answers with stray tags (training template)	the empty think block is removed from the training texts (v2)

DEMO.md holds the jury demo script: what to prepare the day before and 30 minutes before, the run order with talking points, and fallbacks. With the local chat model everything runs offline except the weather and the virtual try-on, which both have fallbacks (calendar season, 2D overlay). The real models still need a dry run on the demo laptop.

13. Local test photos

Our own phone photos (wardrobe and friperie) are the only data from the real DressMe setting, and the only way to measure traditional wear and swimwear. They are the **test set only**. The pipeline is ready; collection starts after the demo (targets: 500–1,000 wardrobe items, 150–300 friperie items, 50–100 traditional garments).

- *LOCAL_PHOTOS.md*: consent and privacy (anonymous contributor ids, no faces, shop owner's permission), how to shoot (one whole item per photo, plain surface, JPEG), and how to fill each label column.
- *src/phase3/map_local.py --init* lists new photos in *labels.csv*; the default run checks every label against *mappings/* and stops with the line number, then writes *local.csv* (ids *lc_*).
- *merge_and_split.py* adds them to *dressme.csv* with *split* and *outfit_split* forced to test; their colours count as real labels.

- Still to do once photos exist: include *local* in the image cache, the embeddings and the evaluation scripts.

14. Key decisions

Decision	Why
EfficientNet for category, sub_category and pattern	team rule "whichever is better": it won all three fields
Keep the gradient-boosting colour model	FashionCLIP vectors are worse on the hand labels
Colour empty below 0.7 confidence	a wrong colour hurts more than none; the user confirms it
Team-set compatibility weights	the data measures them but does not set them; style-only would lose readable reasons
One packed image cache	small-file reads were the training bottleneck
H&M catalogue, then polite Tunisian shops, instead of scraping	the Inditex and H&M sites block automated access; we never try to get around bot protection
Friperie listings approved by an admin	users see only reviewed listings; the agent prepares a sale but never posts it
Remove the background of uploads	the models were trained on white product shots; a bed, rack or person in the photo confuses them
No repeated sub_category / category over its limit	a hard rule, not a lower score: two pairs of jeans is never an outfit
Five agents instead of one assistant	a short prompt and few tools per agent: easier for a small local model, and each can be tested alone
Local chat model, fine-tuned on synthetic chats	no daily quota, works offline; the real tools write the answers, so the model learns our scores and never invents clothes. Team rule: the better model is used (v4)
Explanations only show how an answer was made	they reuse the same code as the answer, so they cannot disagree with it
Local photos for test only	the one honest measure of the real setting; never trained on
Predictions are never labels	<i>predicted_attributes.csv</i> and the <i>predicted</i> field stay separate from ground truth / user input

15. Limits and risks

- Traditional (jebba, kaftan) and swimwear are too rare in the public data to learn; traditional is read as dress. Local photos are needed.
- Street photos remain harder than product shots (category 93% vs 98%).
- Colour, pattern and structure add no measurable signal in the swap test; the team weights cost ~6 AUC points against style alone. The weakest-piece explanation is limited by the same signal.
- PolyVore has no pattern labels, so its patterns are predictions.
- Background removal costs ~3-5 s per photo on the CPU; cloth-seg often misses items lying flat, so it is only used on worn items. Only the cleaned photo is kept.
- The models have not been measured on real phone photos yet (local test set pending).
- Listings depend on shops that may change or block us: only two Tunisian shops run, the Inditex rows are a frozen snapshot, and the H&M catalogue has no prices.
- Virtual try-on depends on free Hugging Face Spaces that can be down or busy (the default one was broken on 2026-10-04); the 2D overlay is the fallback.
- The chat model is tested on synthetic chats from the same generator as its training data; real users will phrase things differently. It is slow on the 4 GB GPU (11-18 s per answer). The Darija it learned is only partly reviewed by a native speaker.
- Many team rules are still marked REVIEW (weights, XAI cuts, resale factors, router keywords, weather thresholds, currency rates).
- Datasets are for non-commercial academic use only and are never redistributed.

16. Next steps

- Retrain the local chat model with the newest Explainer tools (`explain_similarity`), and calibrate the "not sure" cuts on real uploads (Admin > Explainability).
- Native-speaker review of the Darija (*`darija_review.md`*, section 7 is new), then rebuild the chat dataset and retrain once, together with the Shopping advisor's weak spots.
- Try the chat with real users' own phrasing, beyond the synthetic test.
- Dry run on the demo laptop with the real models, background removal, the local chat model and the try-on Spaces (*`DEMO.md`*).
- Collect and label local photos (*`LOCAL_PHOTOS.md`*), then evaluate the classifier, colour model and background removal on them.
- Team session to tune the compatibility weights and close the REVIEW rows.
- User test with the personas; use the admin "model quality" page (correction rates) to see where the models fail in real use.